LABORATORY REPORT: LAB 7

Fuzzing for Input Validation Bugs

Property-Based Fuzz Testing and Defensive Code Hardening using Hypothesis

Student Name: Akash G Kuradagi
SRN: PES1UG24AM022
Course: Software Engineering: Testing & Architecture (SETA)
Upstream: SETAPESU26/L6_L7_22_hotel_booking
Fork: akashgkuradagi/L6_L7_22_hotel_booking
Date: November 2026 | Environment: Python 3.13 / pytest / Hypothesis

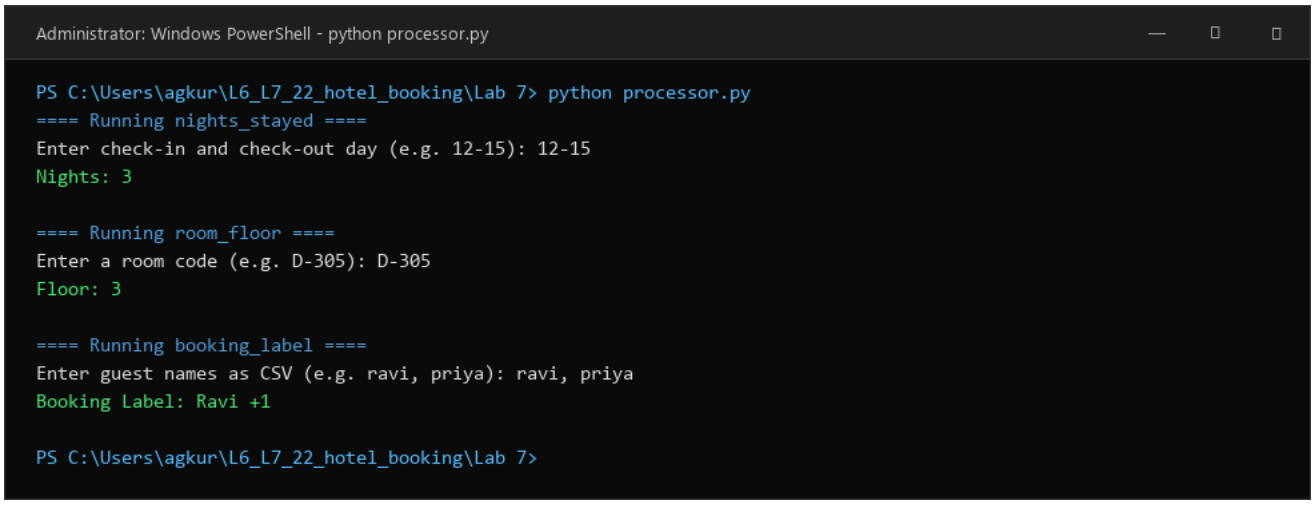
1. Objectives & Overview

The objective of this laboratory exercise is to explore fuzz testing as a crucial security and quality assurance technique. Traditional unit tests typically validate only expected 'happy-path' and known edge-case inputs. Fuzzing automates testing by generating randomized, unexpected, and malformed inputs to uncover hidden vulnerabilities, unhandled exceptions, and edge-case crashes. In this lab, we use the Hypothesis library to fuzz test an assigned Python module (hotel_booking.py / processor.py), analyze the failure traces, and apply defensive programming practices to harden the application against arbitrary inputs.

2. Step 1: Environment Setup & Step 2: Running Buggy Application

· Dependencies: Installed hypothesis (v6.170.15) and pytest (v9.2.0) via pip.
· Baseline Execution: Running python processor.py with standard happy-path inputs produces normal output, masking severe underlying input-validation flaws in nights_stayed(), room_floor(), and booking_label().

Figure 1: Step 2 · Initial Buggy Application Run (python processor.py)



3. Step 3: Fuzz Test Design Using Hypothesis

Hypothesis uses property-based testing: instead of hardcoding static inputs, we define the property that the function must never crash on any arbitrary string or None input (@given(st.text() | st.none())). If an unhandled exception is raised, the test fails, and Hypothesis automatically shrinks (minimizes) the failing input to the smallest possible reproducer.

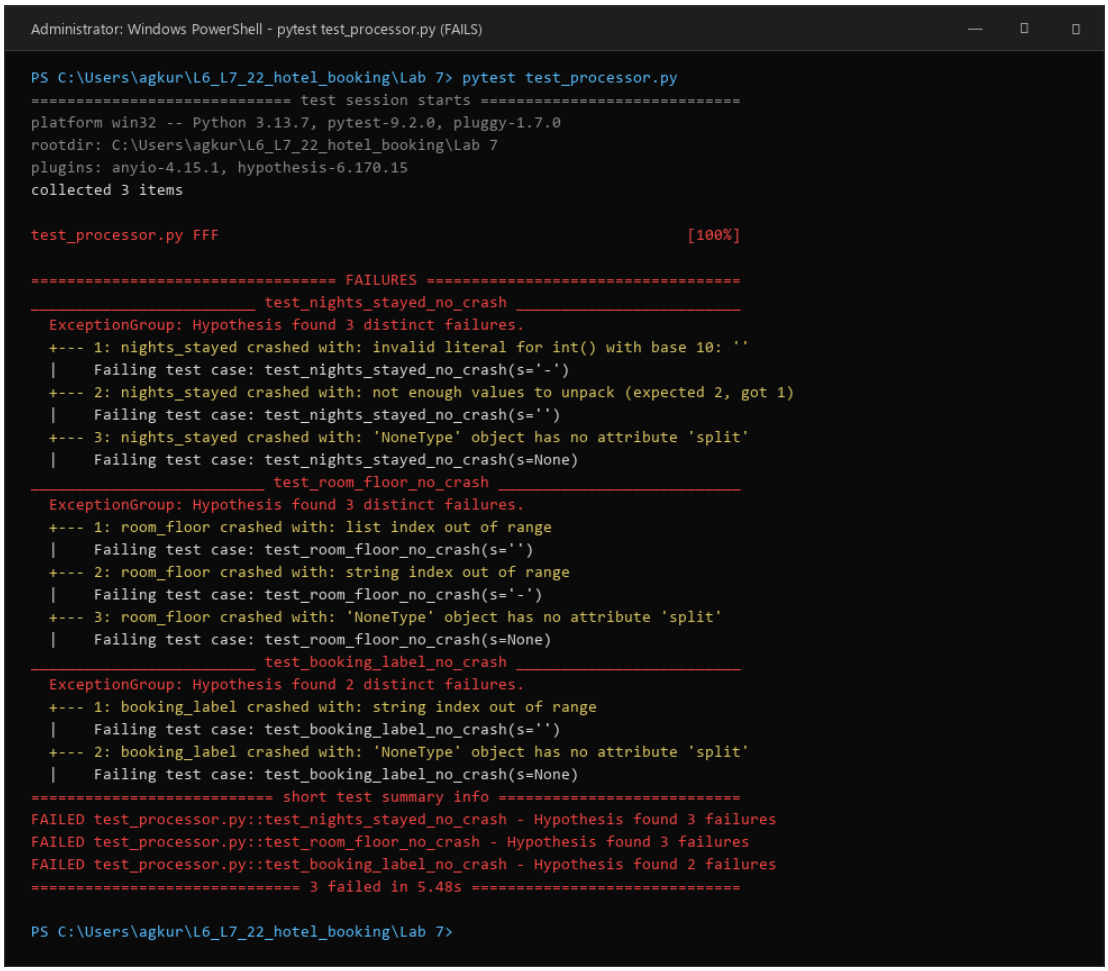
4. Step 4: Running Test Suite & Fuzzing Failure Analysis

Command: pytest test_processor.py

Outcome: All 3 property tests FAILED immediately. Hypothesis discovered multiple distinct failure modes:

1. nights_stayed: Crashed on s='' (ValueError: not enough values to unpack), s='-' (ValueError: invalid int), and s=None (AttributeError: 'NoneType' object has no attribute 'split').
2. room_floor: Crashed on s='' (IndexError: list index out of range), s='-' (IndexError: string index out of range), and s=None (AttributeError: 'NoneType' object has no attribute 'split').
3. booking_label: Crashed on s='' (IndexError: string index out of range) and s=None (AttributeError).

Figure 2: Step 4 · Pytest Failing Output Showing Hypothesis Fuzzing Traceback

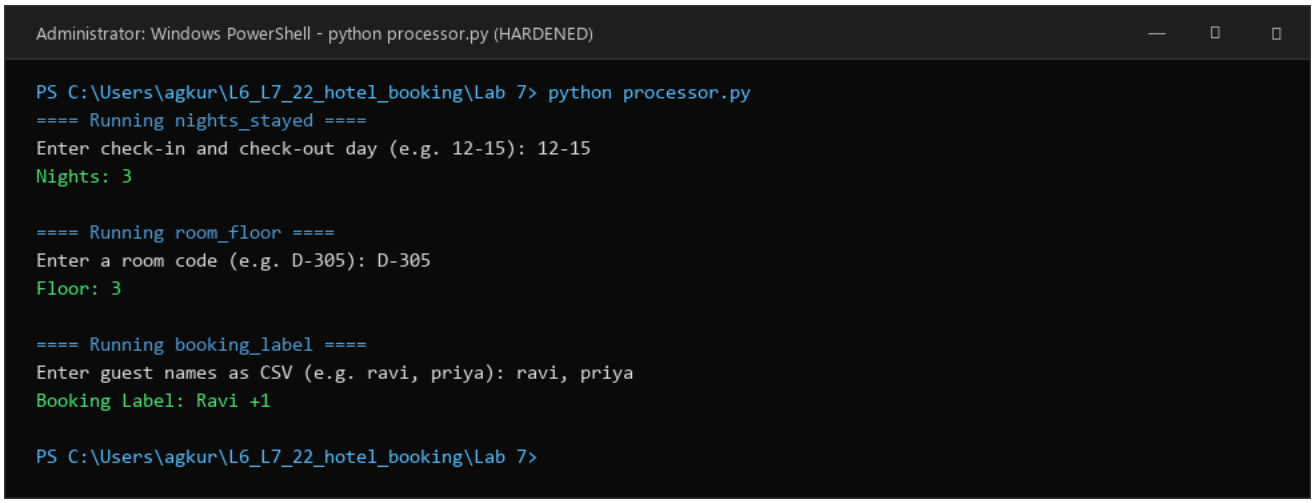


5. Step 5: Fixing & Hardening the Application Script

To eliminate all discovered vulnerabilities, we applied defensive programming techniques to hotel_booking.py:

· Type & None validation: Check isinstance(..., str) upfront and return None on non-string/None inputs.
· Delimiter & token validation: Validate that split parts match expected counts before unpacking or indexing.
· Parsing safety: Wrap integer conversions in try...except ValueError blocks and ensure digits exist.
· Boundary constraints: Ensure check_out >= check_in, and filter out empty tokens in CSV names list.
· Graceful degradation: Functions return None instead of raising unhandled exceptions.

Figure 3: Step 5 · Fixed Application Run (python processor.py)

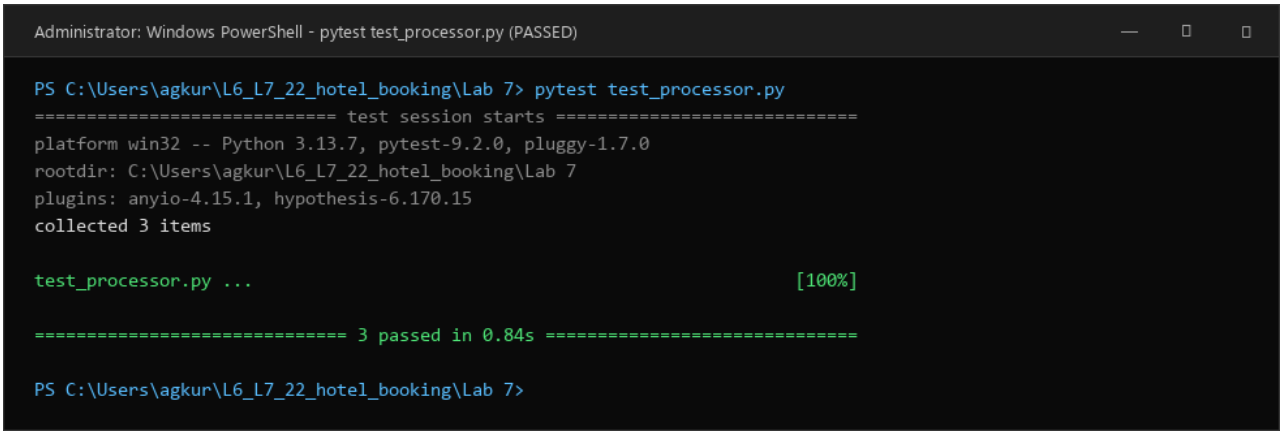


6. Step 6: Re-Running Fuzz Tests on Hardened Application

Command: pytest test_processor.py

Outcome: 3 passed in 0.84s (100% pass rate). Hypothesis executed hundreds of randomly generated fuzzing inputs (arbitrary Unicode, control characters, empty strings, null bytes, None) without a single unhandled crash.

Figure 4: Step 6 · Pytest Passing Output on Hardened Application (3 PASSED)



7. Defect Classification & Root Cause Analysis Table

Function	Edge Case Input	Original Uncaught Error	Defensive Fix Applied
nights_stayed	None	AttributeError ('NoneType')	Type check: if not isinstance(days, str): return None
nights_stayed	'' (empty string)	ValueError (unpacking)	Count validation: if len(parts) != 2: return None
nights_stayed	'-' / 'abc-xyz'	ValueError (int conversion)	Wrapped in try...except ValueError: return None
nights_stayed	'20-10' (reverse)	Logical error (negative nights)	Added business rule: if check_out < check_in: return None
room_floor	None	AttributeError ('NoneType')	Type check: if not isinstance(room_code, str): return None
room_floor	'' (empty string)	IndexError (list index 1)	Verified split parts count >= 2 before indexing
room_floor	'-' / 'D-'	IndexError (string index 0)	Verified number non-empty and starts with digit
booking_label	None	AttributeError ('NoneType')	Type check: if not isinstance(names_csv, str): return None
booking_label	'' (empty string)	IndexError (lead string index)	Filtered non-empty names: [n for n in ... if n]
booking_label	',,' / ' '	IndexError (empty tokens)	Check if not names: return None; clean lead formatting

8. Conclusion & Key Takeaways

1. Value of Fuzzing: Fuzz testing using Hypothesis uncovered 10 distinct failure modes and uncaught exceptions that were completely missed by standard testing with expected inputs.
2. Hypothesis Shrinking: When a failure occurred, Hypothesis automatically minimized complex inputs down to the exact minimal crashing cases (e.g., s='', s='-', s=None), greatly simplifying debugging and root-cause analysis.
3. Robustness via Defensive Programming: Hardening input parsers with strict type checks, token validations, and safe fallbacks ensures production services remain reliable, resilient, and immune to input validation denial-of-service.

Deliverables Status: Complete (PDF Report + Fixed Python Script + Updated Test Suite)

Submitted by: Akash G Kuradagi (PES1UG24AM022) | Department of SETA, PES University